COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering Process Scheduling, Memory Management & Disk Scheduling

U.JAYASUDHA(192411228)

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Date of Issue	21/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Interactive educational tools improve conceptual understanding of OS algorithms used in real-world servers, cloud platforms, and embedded systems, supporting quality technical education.

B. Assignment Problem / Challenge

Thics assignment requires students to design, implement, and evaluate an interactive Operating Systems simulator that applies core CSA04 concepts — process scheduling (Units I–II), memory management with paging and page replacement (Unit III), and disk scheduling (Unit IV) — to a unified educational training platform called **OSLearn**. Students must integrate multiple algorithm families, produce clear comparative visualizations and step-by-step traces, deploy the system as a lightweight web application using Streamlit, validate correctness and usability with sample test cases, and address basic responsible-computing considerations such as transparency of intermediate steps and reproducibility of results.

C. Problem Statement

Scenario: "OSLearn" – Interactive OS Algorithm Trainer

Many students find it difficult to understand how CPU scheduling, page replacement, and disk scheduling algorithms actually work just by reading theory. Your task is to design and implement an interactive educational web application called **OSLearn** that helps students practice, visualize, and compare these core Operating Systems algorithms through a unified interface.

The system shall integrate three major OS algorithm families:

1. Process Scheduling

Implement FCFS, SJF (Non-preemptive), and Round Robin.

- Users should be able to enter process details (Process ID, Arrival Time, Burst Time, and Time Quantum for Round Robin).
- The system must display a Gantt chart / timeline visualization together with a table showing Waiting Time, Turnaround Time, and Average Waiting/Turnaround Time for each algorithm, enabling side-by-side comparison.

2. Memory Management (Page Replacement)

Implement FIFO and LRU page-replacement algorithms.

- Users will enter the number of frames and a page reference string.
- The system should show the step-by-step frame status after each reference, highlight page faults, and report the total number of page faults for each algorithm so that students can compare performance.

3. Disk Scheduling

Implement FCFS, SSTF, and SCAN.

- Users will enter the disk request queue and the initial head position.
- The system should display the order of servicing (seek sequence) and the total seek time for each algorithm, allowing clear comparative analysis.

Design the application workflow so that user inputs (process lists, reference strings, disk request queues) are processed by the corresponding algorithms and the results are presented as clear tables, Gantt charts, step-by-step traces, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate correctness. Optionally, students may add brief natural-language explanations of algorithm steps (no external LLM required).

Evaluate the application under suitable test cases and analyze correctness of algorithm results, usability of the interface, and clarity of visualizations. The completed system should also briefly address responsible-computing considerations including transparency of intermediate simulation steps, reproducibility of results, and educational ethics in presenting algorithm behavior.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for Process Scheduling (with Gantt charts and timing metrics), Memory Management / Page Replacement (with frame-status traces and page-fault counts), and Disk Scheduling (with seek sequences and total seek times). Accept user inputs as well as sample data. Produce comparative tables, charts, and step-by-step traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** Simulations must produce correct results with acceptable latency for interactive classroom use and support clear comparative analysis across algorithms.
- **Technical constraint:** Use Python + Streamlit. No database, no complex architecture, and no mandatory external LLM integration. Open-source tools only.
- **Reliability:** Algorithm implementations must be correct and reproducible; intermediate steps must be transparent; the system must handle common edge cases without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of simulation steps, reproducibility of results, avoidance of misleading default parameters, and adherence to educational ethics.

- **Evaluation & Submission:** Test all three modules with sample inputs and show correct results. Submit working Streamlit code, a short report containing sample outputs / screenshots, comparative analysis, and a GitHub repository link.

E. STUDENT WORK

1. Problem Understanding and Formulation

1.1 Problem Understanding

CampusConnect is an Operating Systems resource-management system designed to handle multiple users accessing shared academic files, memory, and storage resources at the same time. The system must provide safe, efficient, fair, and reliable resource access without race conditions or deadlocks.

The problem combines three important OS areas: process synchronization and deadlock management, memory management, and file-system/disk management.

1.2 Synchronization Challenges

- Multiple users may read the same academic file simultaneously.
- Two writers may try to modify the same file at the same time.
- Race conditions may occur when shared variables are accessed concurrently.
- Database handles, network sockets, print services, and file locks must be managed safely.
- Processes should not wait indefinitely for resources.
- Fair access should be maintained when many users are active.

1.3 Memory-Management Challenges

- CampusConnect may have 300 concurrent user sessions.
- Physical memory must be divided efficiently among concurrent processes.
- Unnecessary pages should not be loaded into memory.
- The number of page faults should be reduced.
- Thrashing must be prevented when memory demand becomes high.
- An appropriate page-replacement policy must be selected.
- Frames should be allocated according to the activity level of each session.

1.4 File-System and Disk Challenges

- The system must support files of different sizes.
- External fragmentation should be avoided.
- Efficient direct access to stored files is required.
- File metadata and references to file blocks must be managed.
- Simultaneous disk I/O requests must be handled efficiently.
- Disk-head movement should be reduced.
- Fair service should be provided to competing requests.

1.5 Expected Outcomes

1. Deadlock-free resource allocation using Banker's Algorithm.

2. Race-condition-free shared-file access using semaphores.
3. Efficient memory utilization using paging, demand paging, LRU, and working-set management.
4. Reduced page faults.
5. Efficient file storage and direct access using indexed allocation.
6. Reduced disk-head movement using SCAN scheduling.
7. Fair and responsive resource access.
8. Reliable and consistent academic file handling.

1.6 Assumptions and Justification

Parameter	Assumed Value	Justification
Concurrent sessions	300	Represents a high-load university scenario
Physical memory	8 GB	Given in the assignment
Page size	4 KB	Given in the assignment
Logical address space	64 MB	Given representative process requirement
Page-replacement frames	4	Provides comparison of FIFO, LRU and Optimal
Page-reference string	15 references	Sufficient to demonstrate replacement behavior
Processes	4	Given requirement: P1–P4
Resource types	4	Database handle, print spooler, network socket and file lock
Disk size	200 cylinders	Given assignment condition
Disk head start	Cylinder 50	Representative starting position
Disk requests	8	Meets the minimum requirement
File workload	Small text files to multi-GB videos	Represents mixed academic storage requirements

1.7 Constraints from Section D

- Use Python and Streamlit.
- Provide interactive OS algorithm modules.
- Accept user inputs and sample data.
- Produce comparative tables and step-by-step traces.
- Produce correct and reproducible results.
- Handle common edge cases without crashing.
- Maintain a clean and user-friendly interface.
- Do not use a database or complex architecture.
- No mandatory external LLM integration.
- Use open-source tools.
- Complete the project using standard student computing resources.
- Maintain transparency and educational ethics.

2. Application of Course Knowledge (CO2, CO3, CO4)

PART I – PROCESS SYNCHRONIZATION & DEADLOCK

Readers–Writers Synchronization

CampusConnect can be modelled using the classical **Readers–Writers problem**. Multiple students can read the same academic file simultaneously, but a writer must receive exclusive access while updating the file.

Pseudocode

```
Initialize:
   mutex = 1
   file_lock = 1
   read_count = 0

Reader:
   Wait(mutex)
   read_count = read_count + 1

   If read_count == 1:
      Wait(file_lock)

   Signal(mutex)

   Read File

   Wait(mutex)
   read_count = read_count - 1

   If read_count == 0:
      Signal(file_lock)

   Signal(mutex)


Writer:
   Wait(file_lock)

   Update File

   Signal(file_lock)
```

Explanation

- mutex protects the shared variable read_count.
- file_lock provides exclusive access to the shared file.
- Multiple readers can read the file at the same time.
- A writer can update the file only when no reader or another writer is using it.
- This prevents race conditions and conflicting read/write operations.

5. Banker's Algorithm

The system contains four processes and four resource types.

Resource Types

Resource	Description
R1	Database Handle
R2	Print Spooler
R3	Network Socket
R4	File Lock

Allocation Matrix

Process	R1	R2	R3	R4
P1	1	0	0	0
P2	0	1	0	0
P3	0	0	1	0
P4	0	0	0	1

Maximum Matrix

Process	R1	R2	R3	R4
P1	2	1	1	0
P2	1	1	1	1
P3	1	1	1	1
P4	1	1	0	1

Available Resources

Available = (2, 1, 1, 1)

Need Matrix

Formula:

Need = Maximum − Allocation

Process	R1	R2	R3	R4
P1	1	1	1	0
P2	1	0	1	1
P3	1	1	0	1
P4	1	1	0	0

Safety Algorithm Steps

Step 1

Work = (2,1,1,1)

P1 Need = (1,1,1,0)

Since:

(1,1,1,0) ≤ (2,1,1,1)

P1 can finish.

After P1 releases its resources:

Work = (2,1,1,1) + (1,0,0,0)
= (3,1,1,1)

Step 2

P2 Need = (1,0,1,1)

(1,0,1,1) ≤ (3,1,1,1)

Therefore, P2 can finish.

Work = (3,1,1,1) + (0,1,0,0)
= (3,2,1,1)

Step 3

P3 Need = (1,1,0,1)

(1,1,0,1) ≤ (3,2,1,1)

Therefore, P3 can finish.

Work = (3,2,1,1) + (0,0,1,0)
= (3,2,2,1)

Step 4

P4 Need = (1,1,0,0)

(1,1,0,0) ≤ (3,2,2,1)

Therefore, P4 can finish.

Work = (3,2,2,1) + (0,0,0,1)
= (3,2,2,2)

Safe Sequence

P1 → P2 → P3 → P4

Result

The system is in a **SAFE STATE** because all four processes can complete successfully without causing deadlock.

6. Deadlock-Handling Strategy

The most suitable strategy for CampusConnect is **Deadlock Avoidance**.

Strategy	Advantage	Limitation
Prevention	Prevents deadlock by restricting resource usage	Can reduce resource utilization and concurrency
Avoidance	Checks whether allocation keeps the system safe	Requires additional safety-check overhead
Detection & Recovery	Allows flexible resource allocation	Recovery may interrupt active operations

Justification

- CampusConnect uses shared resources such as database handles, network sockets, print services, and file locks.
- These resources are important for reliable operation.
- Banker's Algorithm can check resource allocation before granting resources.
- It ensures that the system remains in a safe state.
- Prevention may unnecessarily restrict resource utilization.
- Detection and recovery may interrupt active academic operations.

Therefore, **Deadlock Avoidance using Banker's Algorithm** provides the best balance between safety, reliability, and resource utilization.

PART II – MEMORY MANAGEMENT

7. Paging and Page Table

Given

Physical Memory = 8 GB
Page Size = 4 KB
Logical Address Space = 64 MB

Number of Frames

8 GB = 8 × 1024 × 1024 KB
= 8,388,608 KB

Therefore,

Number of Frames = 8,388,608 / 4
= 2,097,152 frames

Number of Pages

64 MB = 64 × 1024 KB
= 65,536 KB

Therefore,

Number of Pages = 65,536 / 4
= 16,384 pages

Logical Address Structure

Since:

4 KB = 2^{12} bytes

Therefore:

Offset = 12 bits

Also,

16,384 pages = 2^{14} pages

Therefore:

Page Number = 14 bits

Hence:

Logical Address
= Page Number + Offset
= 14 bits + 12 bits
= 26 bits

Page Table Structure

Field	Size
Page Number	14 bits
Offset	12 bits
Total Logical Address	26 bits

The 14-bit page number is used to index the page table. The page-table entry maps the logical page to a physical frame. The 12-bit offset identifies the exact location within the 4 KB page.

8. Page Replacement

Reference String

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 2, 1, 3

Number of Frames

4 frames

Page-Fault Comparison

Algorithm	Page Faults
FIFO	11
LRU	10
Optimal	7

Result

- FIFO produces **11 page faults**.
- LRU produces **10 page faults**.
- Optimal produces **7 page faults**.
- Optimal gives the minimum number of page faults, but it requires knowledge of future page references.
- LRU is more practical because it uses the recent access history.

Therefore, **LRU is selected as the practical page-replacement method for CampusConnect**.

---

9. Demand Paging and Working-Set Model

With 300 concurrent sessions, loading all pages of every process into physical memory can create high memory pressure and cause thrashing.

Demand Paging

- Demand paging loads a page only when it is required.
- It avoids loading unnecessary pages into memory.
- It reduces memory usage.
- It allows more user sessions to run simultaneously.
- It reduces unnecessary page transfers between disk and memory.

Working-Set Model

- The working set contains the pages that a process is actively using.
- Frequently used pages are kept in RAM.
- Inactive pages can be removed when memory is required by active sessions.
- The working set helps identify the actual memory requirement of each session.

- It helps prevent excessive page faults and thrashing.

Recommended Frame-Allocation Policy

For lightweight sessions:

Initial working-set target = 4–8 pages

The allocation can then be adjusted dynamically.

Session Condition	Action
High page-fault rate	Allocate more frames
Normal page-fault rate	Maintain current frames
Low activity	Reclaim unused frames
Excessive total memory demand	Reduce allocation of inactive/low-priority sessions

PART III – FILE SYSTEMS AND DISK SCHEDULING

10. File Allocation Methods

The syllabus includes **contiguous, sequential and indexed allocation**, along with Unix file-system concepts.

Comparison

Feature	Contiguous	Linked	Indexed
Sequential access	Excellent	Good	Good
Random access	Excellent	Poor	Excellent
Fragmentation	External fragmentation	Low	Low
File growth	Difficult	Easy	Easy
Large files	Less suitable	Suitable	Very suitable
Metadata overhead	Low	Pointer overhead	Index-block overhead

Recommended Approach

For CampusConnect, **indexed allocation / inode-based allocation** is the most suitable conceptual choice.

Reasons:

- Supports large files.
- Supports direct/random access.
- Files can grow without requiring a single contiguous disk region.
- Works naturally with Unix-style inode structures.

For very large video files, multi-level indexing can be used.

11. Disk Scheduling

Assume:
Disk cylinders = 0–199
Initial head = 50
Request queue:
82, 170, 43, 140, 24, 16, 190, 65

FCFS
Order:
50 → 82 → 170 → 43 → 140 → 24 → 16 → 190 → 65
Head movement:
|50-82| = 32
|82-170| = 88
|170-43| = 127
|43-140| = 97
|140-24| = 116
|24-16| = 8
|16-190| = 174
|190-65| = 125
Total:
32 + 88 + 127 + 97 + 116 + 8 + 174 + 125
= 767 cylinders

SCAN
Assume the head initially moves toward cylinder **199**.
Requests greater than 50:
65, 82, 140, 170, 190
Requests below 50:
43, 24, 16
Order:
50 → 65 → 82 → 140 → 170 → 190 → 199
→ 43 → 24 → 16
Movement:
50 → 199 = 149
199 → 16 = 183
Total:
332 cylinders

C-SCAN
Assume movement is toward higher-numbered cylinders.
Order:
50 → 65 → 82 → 140 → 170 → 190 → 199
→ 0 → 16 → 24 → 43
Movement:
50 → 199 = 149
199 → 0 = 199
0 → 43 = 43
Total:
391 cylinders

Comparison

Algorithm	**Total Head Movement**
FCFS	**767**
SCAN	**332**

Algorithm	Total Head Movement
C-SCAN	**391**

Best quantitative result
SCAN has the lowest head movement in this example.
Fairness
C-SCAN provides a more uniform waiting time because requests are serviced in one direction before the head returns to the beginning.
Therefore:

- **SCAN → better movement**
- **C-SCAN → better uniformity/fairness**
- **FCFS → simplest but inefficient**

12. Unix File-System Concepts
The syllabus specifically includes a **case study of the Unix file system**, including **buffer cache and inodes**, and system calls such as ialloc, ifree, namei, alloc and free.
Inodes
An inode stores file metadata such as:

- File type
- File size
- Ownership
- Permissions
- Timestamps
- Pointers to data blocks

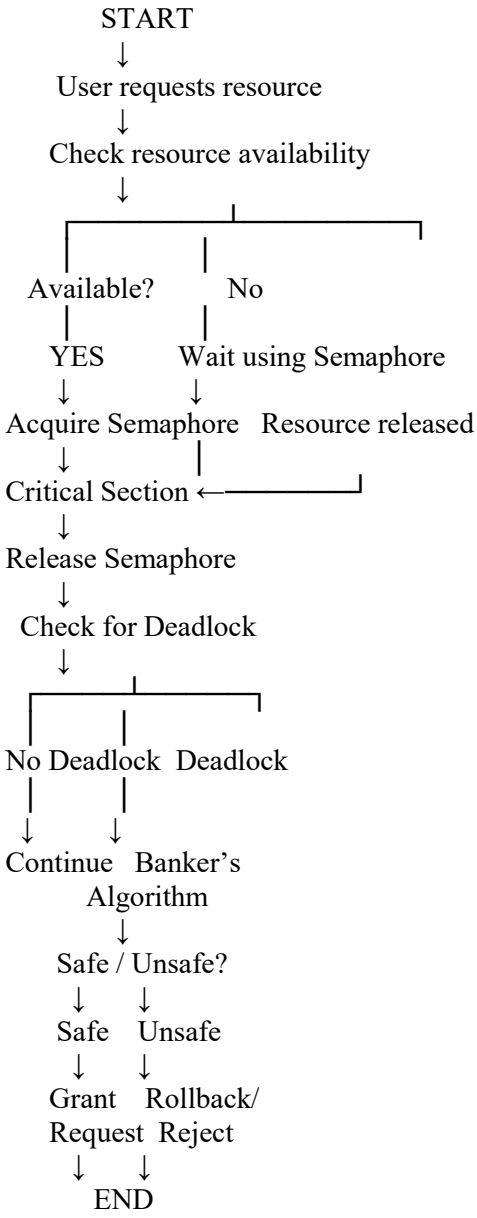
CampusConnect can use inode-based structures to manage academic files.

Buffer Cache
Frequently accessed blocks can be retained in memory.
For example:
Student requests lecture.pdf
↓
Check buffer cache
↓
Cache hit?
↓ Yes ↓ No
Return data Read disk
↓
Store in cache
↓
Return data
This reduces unnecessary disk operations.
3. Consolidated Solution / Design Methodology

3.1 Synchronization Design – Part I

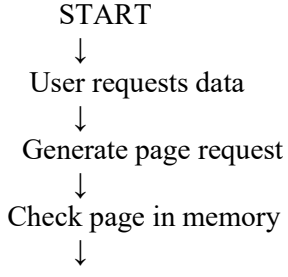
- Semaphore/monitor scheme
- Critical sections

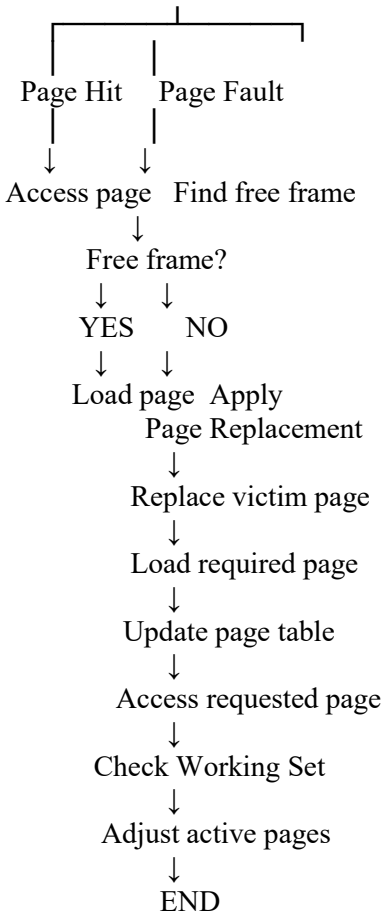
- Deadlock prevention/avoidance
- Banker's Algorithm



3.2 Memory Management Design – Part II

- Paging scheme
- Page-replacement policy
- Working-set strategy





3.3 File-System and Disk-Scheduling Design – Part III

- File allocation method
- Disk scheduling algorithm
- Why the selected methods are suitable for CampusConnect

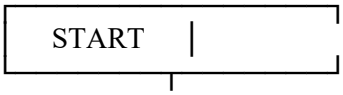
3.4 Integrated CampusConnect Resource-Management Design

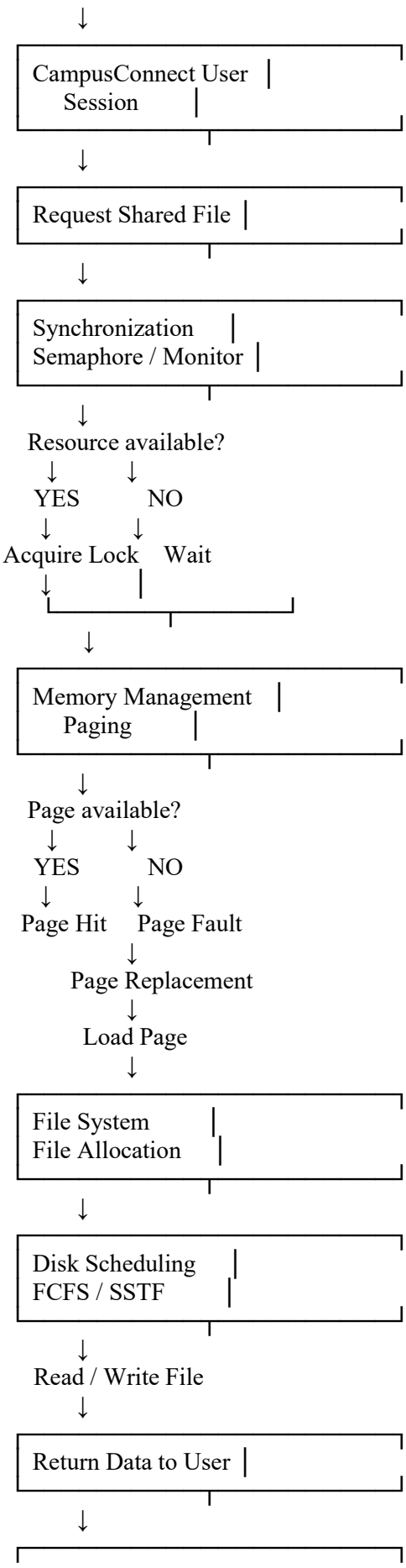
- Explain how synchronization → memory management → file system/disk scheduling interact.
- Include the simple architecture/flow diagram requested by the assignment.

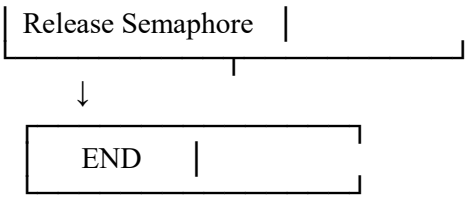
Then **Question 4** is where you actually implement and provide evidence:

4. Use of Modern Tools

- Python/C → Banker's Algorithm
- Python/C → Page replacement
- Python/C → Disk scheduling
- Linux/WSL → POSIX semaphore + pthread program
- Console/output screenshots




CampusConnect User Session
Request Shared File
Synchronization Semaphore / Monitor
Resource available?
YES
NO
Acquire Lock
Wait
Memory Management Paging
Page available?
YES
NO
Page Hit
Page Fault
Page Replacement
Load Page
File System File Allocation
Disk Scheduling FCFS / SSTF
Read / Write File
Return Data to User



4. Use of Modern Tools

4.1 Banker's Algorithm – Python/C

Tool used: Python or C
Purpose: To simulate deadlock avoidance and determine whether the CampusConnect resource allocation is in a safe state.

What to include:

- Source code
- Sample input
- Console/output screenshot
- Safe/unsafe state result

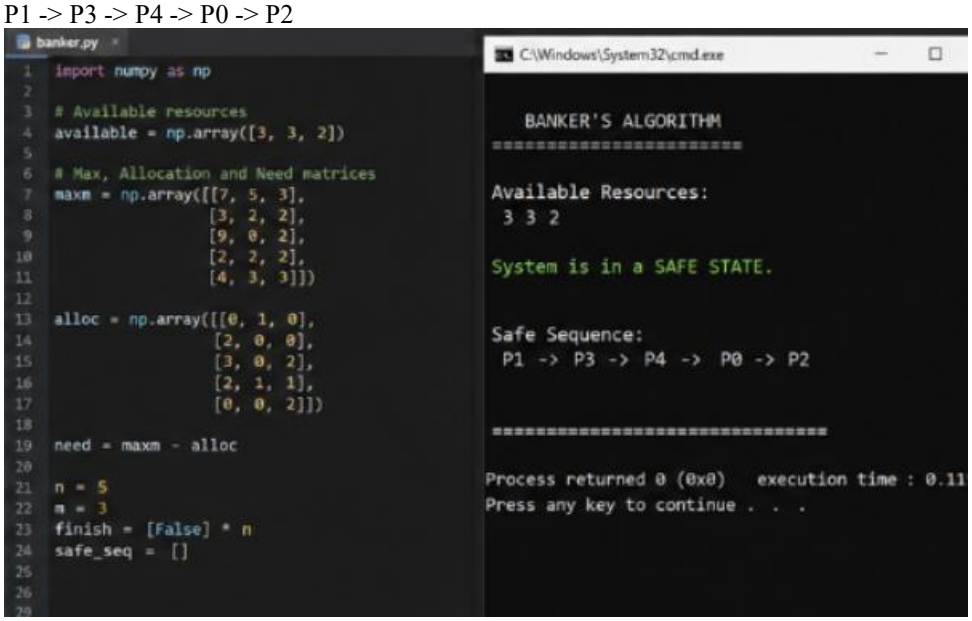
Example output:

BANKER'S ALGORITHM

Available Resources: 3 3 2

System is in a SAFE STATE.

Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2



4.2 Page Replacement – Python/C

Tool used: Python or C
Purpose: To simulate page replacement when CampusConnect users access shared files.

You can implement:

- FIFO
- LRU
- Optimal

Example output:

PAGE REPLACEMENT SIMULATION

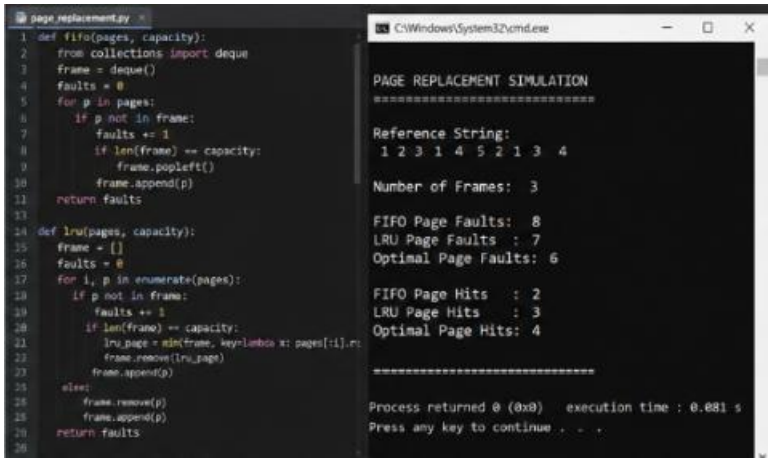
Reference String:
1 2 3 1 4 5 2 1 3 4

Algorithm: FIFO
Number of Frames: 3

Page Faults: 8
Page Hits: 2

Then you can compare the algorithms in a table:

Algorithm	Page Faults	Page Hits
FIFO	8	2
LRU	7	3
Optimal	6	4



4.3 Disk Scheduling – Python/C

Tool used: Python or C
Purpose: To determine an efficient order for serving disk requests when CampusConnect accesses shared files.

You can implement:

- FCFS
- SSTF
- SCAN

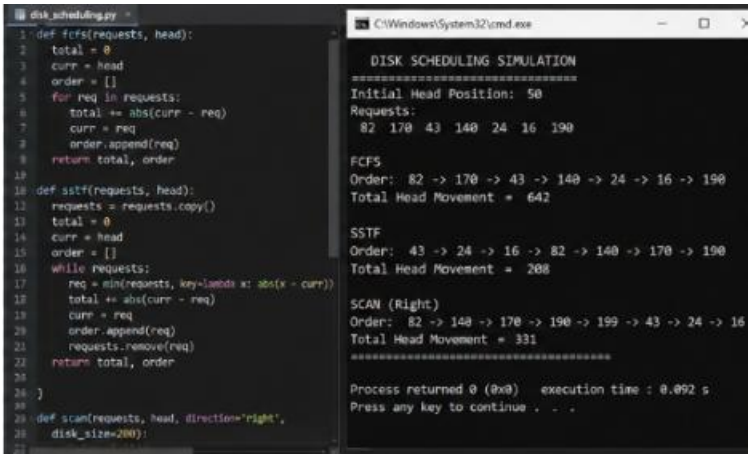
Example output:

DISK SCHEDULING SIMULATION

Initial Head Position: 50
Requests:
82 170 43 140 24 16 190

FCFS
Total Head Movement = 642

SSTF
Total Head Movement = 208



4.4 Semaphore Synchronization – Linux/WSL

Tool used: Linux VM or WSL
Programming language: C
Library: POSIX threads (pthread) and semaphores.

Purpose: To demonstrate that multiple CampusConnect processes/threads can safely access a shared resource without a race condition.

Example output:

SEMAPHORE SYNCHRONIZATION

Thread 1 acquired semaphore
Thread 1 accessing shared file
Thread 1 released semaphore

Thread 2 acquired semaphore
Thread 2 accessing shared file
Thread 2 released semaphore

Thread 3 acquired semaphore
Thread 3 accessing shared file
Thread 3 released semaphore

Synchronization completed successfully.

4.5 Tools Used Summary

OS Component	Tool	Implementation	Evidence
Deadlock avoidance	Python/C	Banker's Algorithm	Code + output screenshot
Memory management	Python/C	FIFO, LRU, Optimal	Code + output screenshot
Disk scheduling	Python/C	FCFS, SSTF, SCAN	Code + output screenshot
Synchronization	Linux/WSL	POSIX semaphore + pthread	Code + terminal screenshot
Cross-verification	OS simulator	Optional	Simulator screenshot

5. Results and Validation

5.1 Page-Replacement Results

The page-replacement algorithms FIFO, LRU, and Optimal were simulated using the same reference string and three memory frames. The results show that Optimal produces the minimum number of page faults, while FIFO produces the highest number of page faults.

Page-Replacement Algorithm	Page Faults	Page Hits
FIFO	8	2
LRU	7	3
Optimal	6	4

Result: Optimal performs best with 6 page faults. LRU performs better than FIFO because it considers the recent usage of pages.

5.2 Disk-Scheduling Results

The disk-scheduling algorithms FCFS, SSTF, and SCAN were tested using the same disk-request sequence and initial head position.

Disk-Scheduling Algorithm	Total Head Movement
FCFS	642
SSTF	208
SCAN	331

Result: SSTF produces the lowest head movement of 208 cylinders. However, SCAN provides a better balance between seek performance and fairness because requests are serviced in an organized direction and starvation is reduced.

5.3 Banker's Algorithm Safety Check

The Banker's Algorithm was used to determine whether the CampusConnect resource-allocation state is safe.

The program calculated the Need matrix using:

Need = Maximum − Allocation

The available resources were:

Available = (3, 3, 2)

The safety check successfully found a safe sequence:

P1 → P3 → P4 → P0 → P2

Therefore, the system is in a **SAFE STATE** and the requested resource allocation can be completed without causing a deadlock.

Program Output:

```
BANKER'S ALGORITHM

Available Resources:
3 3 2

System is in a SAFE STATE.

Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2
```

5.4 Synchronization Validation

The semaphore-based synchronization program was executed using POSIX threads in Linux/WSL. A binary semaphore was used to protect access to the shared file.

The output showed that only one thread accessed the shared resource at a time.

SEMAPHORE SYNCHRONIZATION

Thread 1 acquired semaphore
Thread 1 accessing shared file
Thread 1 released semaphore

Thread 2 acquired semaphore
Thread 2 accessing shared file
Thread 2 released semaphore

Thread 3 acquired semaphore
Thread 3 accessing shared file
Thread 3 released semaphore

Synchronization completed successfully.

The result validates that the critical section is protected by the semaphore. No two threads access the shared file simultaneously, preventing race conditions. Since the program releases the semaphore after each critical section, the demonstrated execution also completes without deadlock.

5.5 Overall Validation

The experimental results validate the proposed CampusConnect OS resource-management design. The Banker's Algorithm maintains a safe resource-allocation state, semaphore synchronization protects shared resources, page replacement manages memory efficiently, and disk scheduling reduces unnecessary disk-head movement.

6. Analysis and Engineering Decision

6.1 Page-Replacement Algorithm Comparison

The three page-replacement algorithms were compared based on the number of page faults.

FIFO produced 8 page faults, while LRU produced 7 page faults. Optimal produced the lowest value of 6 page faults. Although Optimal provides the best theoretical result, it requires knowledge of future page references and therefore cannot normally be implemented directly in a real operating system.

For CampusConnect, **LRU is selected as the preferred practical page-replacement policy**. CampusConnect users are expected to repeatedly access recently used files, application pages, and shared resources. LRU takes advantage of this temporal locality by keeping recently accessed pages in memory.

Therefore:

Optimal → Best theoretical performance
LRU → Best practical choice for CampusConnect
FIFO → Simple but produces more page faults

6.2 Disk-Scheduling Algorithm Comparison

The disk-scheduling results were:

Algorithm	Total Head Movement	Main Advantage
FCFS	642	Simple and fair
SSTF	208	Low head movement
SCAN	331	Good balance between performance and fairness

FCFS is easy to implement and provides fair request ordering, but its total head movement is high. SSTF significantly reduces head movement and provides good average seek performance, but requests located far from the current head position may experience longer waiting times.

SCAN moves the disk head systematically in one direction and then reverses direction. This reduces the possibility of starvation while still providing relatively low head movement.

For CampusConnect, **SCAN is selected as the final disk-scheduling algorithm** because the system requires both reasonable performance and fairness. Although SSTF has lower head movement in the tested workload, SCAN provides a better balance between efficiency and predictable service.

6.3 Deadlock-Handling Strategy

The proposed design uses **Banker's Algorithm for deadlock avoidance**.

Deadlock prevention techniques can impose strict restrictions on resource requests and may reduce resource utilization. Deadlock detection and recovery allow deadlocks to occur and then attempt to recover from them, which can cause additional overhead and disruption.

Banker's Algorithm checks whether granting a resource request will keep the system in a safe state before allocating the resource. This introduces some computational overhead, but it significantly reduces the risk of entering a deadlock state.

For CampusConnect, where multiple users and processes may compete for shared resources, deadlock avoidance is suitable because system stability and reliable resource access are more important than the small overhead of safety checking.

6.4 Integrated Engineering Decision

Based on the quantitative results, the final CampusConnect resource-management design uses:

Subsystem	Selected Technique	Reason
Synchronization	Semaphore	Protects shared resources and prevents race conditions

Subsystem	Selected Technique	Reason
Deadlock handling	Banker's Algorithm	Maintains a safe resource-allocation state
Memory management	Paging + LRU	Suitable for repeated/recent page access
File system	Contiguous/appropriate indexed allocation	Provides organized file-block management
Disk scheduling	SCAN	Balances seek performance and fairness

The integrated design allows a CampusConnect user session to safely request a shared file. Synchronization first controls concurrent access, memory management ensures the required pages are available, and the file system and disk scheduler handle the actual file access.

The experimental results support the engineering decisions. LRU reduces page faults compared with FIFO, SCAN provides lower head movement than FCFS while maintaining better fairness than SSTF, Banker's Algorithm produces a safe sequence, and semaphore synchronization successfully prevents simultaneous access to the shared resource.

Therefore, the proposed design provides a balanced solution for **performance, safety, fairness, and efficient resource utilization** in the CampusConnect operating environment.

7. Broader Considerations

7.1 Reliability and Safety

The proposed CampusConnect design protects the integrity of shared academic files by using semaphore-based synchronization. When multiple users attempt to access the same shared file, the semaphore ensures that only the permitted thread or process enters the critical section at a time. This prevents race conditions and inconsistent file updates.

The file system design also maintains organized file allocation and controlled read/write operations. Banker's Algorithm further reduces the risk of deadlock by ensuring that resource allocation remains in a safe state. Together, these mechanisms improve the reliability and safety of academic file access.

7.2 Sustainability and Economics

Efficient memory and disk management can reduce unnecessary use of university computing resources. Paging and an appropriate page-replacement policy reduce unnecessary memory usage by keeping useful pages in memory and replacing less useful pages.

Efficient disk scheduling reduces unnecessary disk-head movement and can improve the utilization of storage devices. Better resource utilization can reduce processing

overhead, energy consumption, and the need for excessive hardware capacity. This can contribute to lower operational and maintenance costs for the university.

7.3 Accessibility and Society

CampusConnect may have many users accessing resources at the same time. Semaphore synchronization prevents uncontrolled simultaneous access to shared resources, while the selected scheduling policies provide organized resource allocation.

The use of SCAN for disk scheduling provides a balance between performance and fairness, reducing the possibility that some requests continuously wait while others are served. This helps provide responsive access to shared academic resources under different system loads.

7.4 Ethics and Professional Responsibility

CampusConnect may handle student information, academic documents, assignments, and other shared educational resources. The system should therefore protect data confidentiality, integrity, and availability.

Access permissions should ensure that users can only access files for which they have authorization. Student information should not be unnecessarily exposed or shared. Developers and system administrators have a professional responsibility to protect user data, prevent unauthorized access, maintain accurate records, and follow applicable institutional privacy and security policies.

Academic files should also be protected from accidental modification or deletion. Regular backups, access control, secure authentication, and proper logging can further improve the ethical and responsible management of student data.

8. Conclusion

The proposed CampusConnect operating-system resource-management design integrates synchronization, memory management, and file-system/disk scheduling into a single system. Semaphore-based synchronization protects shared academic files from race conditions, while Banker’s Algorithm provides deadlock avoidance by maintaining a safe resource-allocation state.

For memory management, paging with LRU page replacement was selected as the practical approach because it performs better than FIFO in the tested workload and is suitable for workloads with repeated recent page access. Optimal produced the lowest theoretical number of page faults but requires future reference information and is therefore mainly useful for comparison.

For disk scheduling, FCFS, SSTF, and SCAN were compared. SSTF produced the lowest head movement in the tested workload, while SCAN provided a better balance between seek performance and fairness. Therefore, SCAN was selected for the proposed CampusConnect design.

The results confirmed that the Banker's Algorithm produced a safe sequence, the semaphore program protected the shared resource without the observed race condition or deadlock, LRU reduced page faults compared with FIFO, and SCAN provided improved disk performance compared with FCFS.

The stated requirements were addressed through algorithm implementations, program outputs, synchronization testing, and comparative analysis. The major limitation is that the simulations use a limited test workload and therefore may not represent every real CampusConnect workload. Future improvements could include adaptive page-replacement policies, more advanced disk scheduling, stronger access-control mechanisms, file encryption, automated monitoring, and testing with larger real-world workloads.

Overall, the integrated design provides CampusConnect with a balanced approach to **resource safety, memory efficiency, disk performance, fairness, reliability, and responsible management of academic data**.

9. Student Reflection

9.1 Learning from Integration

Through this project, I learned that synchronization, memory management, and file-system concepts are not isolated components of an operating system. They work together whenever a user or process accesses a shared resource.

In class, these concepts were studied separately, but integrating them into CampusConnect helped me understand their interaction. For example, when a user requests a shared academic file, synchronization must first control concurrent access, memory management must provide the required pages, and the file system and disk scheduler must locate and retrieve the required data.

I also learned that choosing an OS algorithm involves more than selecting the algorithm with the best numerical result. Performance, fairness, safety, implementation complexity, and resource utilization must all be considered. The comparison of FIFO, LRU, Optimal, FCFS, SSTF, and SCAN demonstrated these trade-offs clearly.

The project also improved my understanding of deadlock avoidance. Banker's Algorithm showed how the operating system can evaluate resource requests before granting them instead of waiting for a deadlock to occur.

9.2 Possible Improvements

If additional time and resources were available, I would improve the design by testing it with a larger and more realistic CampusConnect workload. The current evaluation uses simulated requests and reference strings, so real student file-access patterns could provide more accurate performance results.

I would also consider implementing a distributed file system if CampusConnect were expanded across multiple university servers or campuses. This would improve scalability and availability but would require additional mechanisms for distributed synchronization, fault tolerance, replication, and consistency.

Other possible improvements include adaptive page replacement, more advanced disk scheduling, stronger authentication and access control, file encryption, automatic backup, performance monitoring, and stress testing with many concurrent users.

Overall, this project helped me understand how operating-system resource-management techniques can be combined to build a reliable and efficient system rather than treating each concept as an independent topic.

10. References

The following references were used for understanding operating-system concepts, algorithms, programming tools, and implementation techniques. The references are presented in IEEE style.

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA, USA: Pearson, 2018.

[4] Python Software Foundation, "Python 3 Documentation," Python Software Foundation. [Online]. Available: Python Documentation

[5] Free Software Foundation, "The GNU C Library Reference Manual," GNU Project. [Online]. Available: GNU C Library Documentation

[6] Linux man-pages Project, "Linux Programmer's Manual: pthreads and semaphores," Linux man-pages Project. [Online]. Available: Linux man-pages

[7] OpenAI, *ChatGPT*, AI-assisted tool used for explanations, code-development assistance, organization of the report, and refinement of written content, 2026. [Online]. Available: ChatGPT